

MARKET MOOD FORECASTING

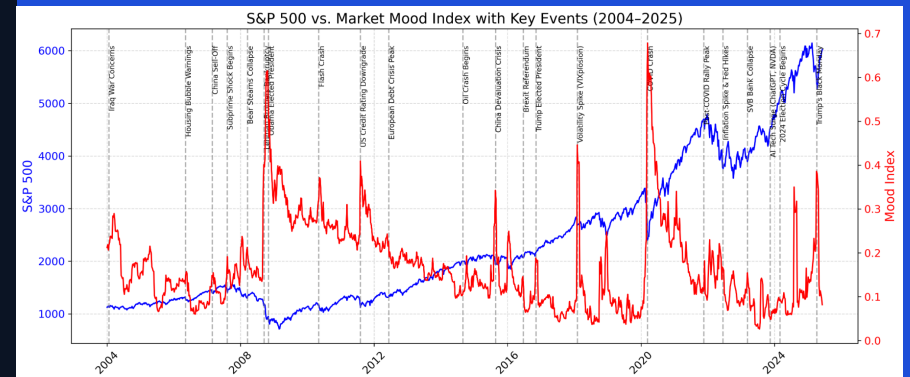
Leakage-safe weekly S&P 500 downside-risk forecasting

A portfolio-grade ML system rebuilt around time-aware validation, historical feature engineering, explainability, and a working Gradio application.

Hotfix v1.1.2

Interpretable ML

App + SHAP



Executive summary

The value of v1.1. is trustworthy process, not exaggerated accuracy.

0.53

ROC AUC

Slight lift over random;
realistic after leakage
removal.

0.45

PR AUC

More informative for the
positive drop class.

0.57

Best F1

Optimized decision
threshold under
imbalance.

0.41

Threshold

Early warning boundary;
not a trading trigger.

Senior DS read

The model is modest because the leakage hotfix removed unrealistic information. That is a credibility gain, not a failure.

Core positioning

This is a risk-scoring case study: time-aware validation, interpretable modeling, transparent limitations, and a deployable app.

Debugging the leakage signal

Suspiciously good results were treated as a risk signal, not as a success.

1. Symptom

Pre-hotfix validation looked unrealistically strong: ROC AUC about 0.65. That did not match the domain.

2. Investigation

I checked target-adjacent columns, future-like names, feature timing, and whether the model indirectly saw next-week information.

3. Rebuild

v1.1.2 removed leakage risks, rebuilt the feature matrix, reran modeling and SHAP, and aligned the deployed app artifact.

Senior DS signal

I did not hide the weaker post-hotfix score. I used it to demonstrate disciplined model rebuilding, leakage awareness, and honest evaluation.

Concrete decision scenario

A model is useful only when it supports a repeatable decision workflow.

User

Portfolio analyst, macro analyst, or risk team reviewing weekly downside risk.

Decision

Flag elevated market-risk weeks for additional review; do not automate trades.

Action rule

Watch: probability > 0.41. Strong alert: probability > 0.60 for 2 consecutive weeks.



Business value: a transparent weekly risk dashboard that helps humans prioritize attention when volatility and sentiment conditions deteriorate.

Data and signal sources

Multiple weak signals are engineered into a time-aware feature matrix.

Market

S&P 500 returns, lagged movement, rolling stability.

Volatility

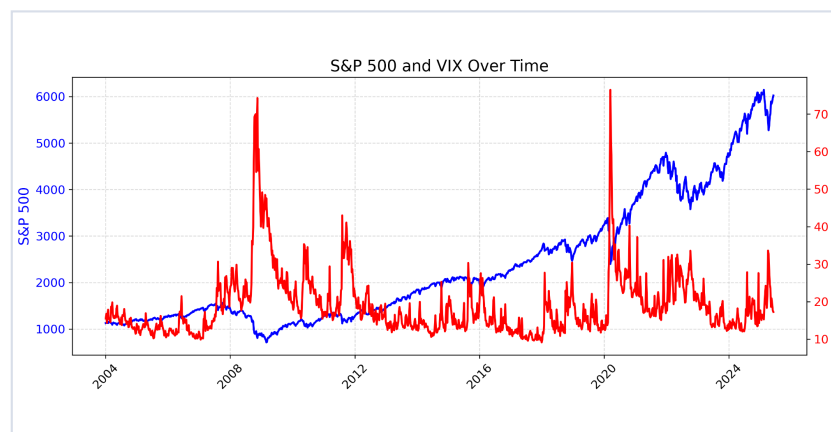
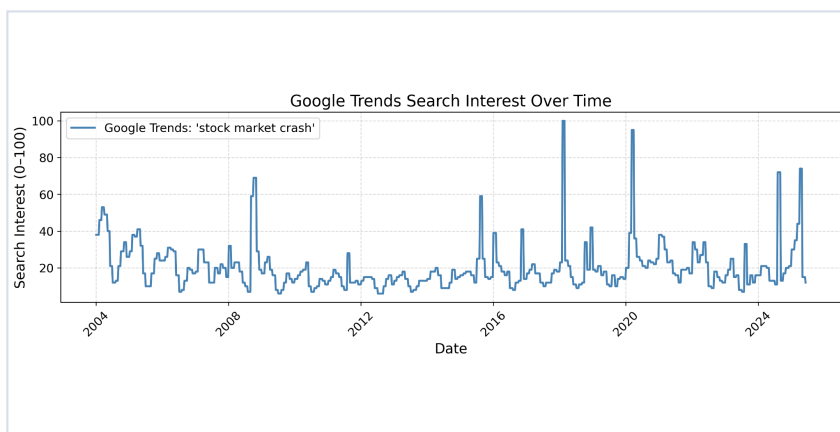
VIX changes, rolling lag statistics, stability ratios.

Sentiment

Google Trends and sentiment index features.

Macro

Unemployment and slower-moving market context.



Takeaway: inputs reflect recognizable stress regimes, but no single signal is strong enough to justify aggressive claims. Dataset: weekly observations (~2004 - 2025), ~950 samples, 32 engineered features across market, volatility, sentiment, and macro sources.

Leakage control is the core differentiator

The v1.1 rebuild treats leakage prevention as the product requirement.

Allowed inputs

- Lagged returns
- Lagged VIX changes
- Rolling windows using historical observations
- Macro values available at prediction time
- Past-only sentiment aggregates

Blocked inputs

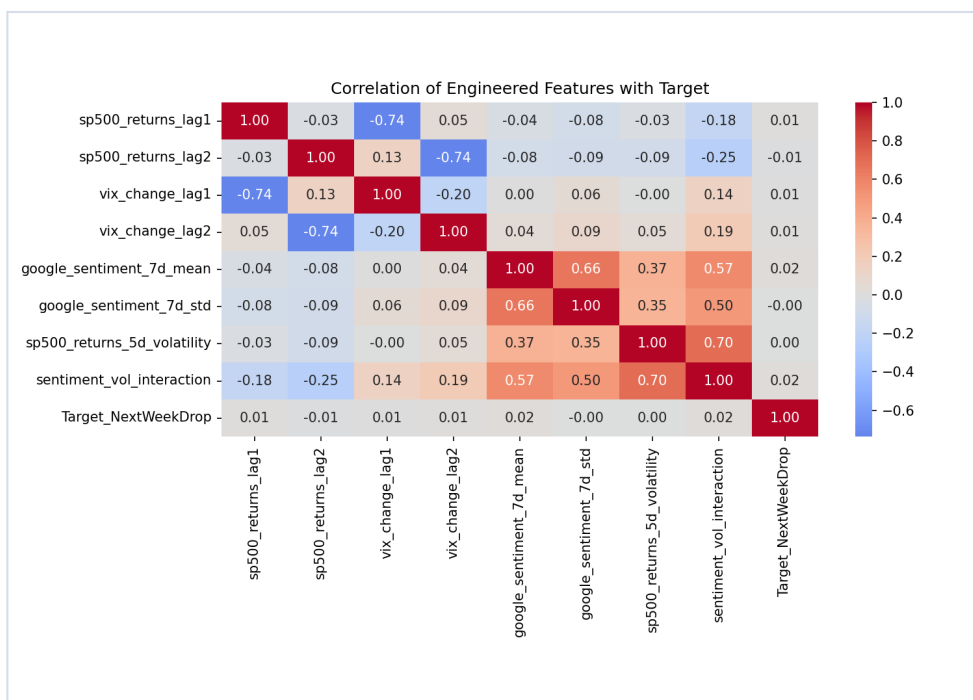
- Target_NextWeekDrop
- Mood_Zone and Mood_Zone_Cat
- next / future / lead / t+ patterns
- target-derived categories
- raw target-adjacent leakage fields

Defense in depth

Leakage checks are enforced twice: first in the notebook pipeline, then again in the Gradio app metadata/guardrail layer. This makes the deployment harder to misuse even if a bad feature accidentally appears upstream.

Feature engineering reveals a hard but plausible problem

The feature matrix is economically coherent; direct target correlation is intentionally weak.



- VIX changes are inversely related to S&P returns, which is economically plausible.
- Sentiment and volatility features cluster, especially the interaction term.
- Target correlations are near zero because weekly market drops are noisy events.
- Do not over-prune from Pearson correlation alone; threshold and interaction effects can still matter.

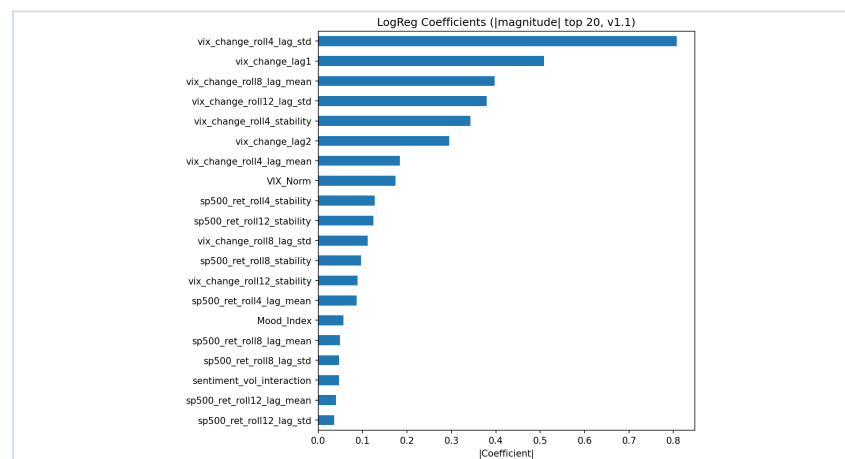
Implication: the model should be framed as risk scoring, not deterministic prediction.

Simple model chosen intentionally

A controlled baseline is better than a complex model with fragile or inflated performance.



- Interpretable coefficients are easy to inspect and defend.
- A linear baseline reduces overfitting risk in a high-noise domain.
- Threshold optimization is explicit, not hidden inside the app.
- Alternative models (including XGBoost) were tested but did not show stable improvement under time-aware validation; Logistic Regression was selected for robustness and interpretability.



Performance is modest - and that is the honest result

The model beats random only slightly, which is realistic for weekly market forecasting.

Validation approach: chronological holdout (leakage-safe)
Known limitation: single split may be unstable in time-series
Next step (validation): rolling / walk-forward CV for robustness

0.50

Random baseline
Expected ROC AUC
from no skill classifier.

0.53

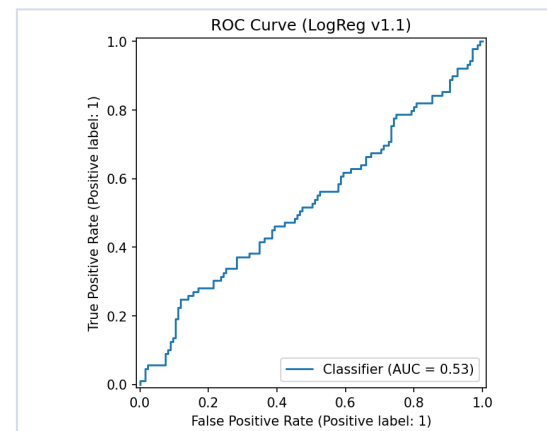
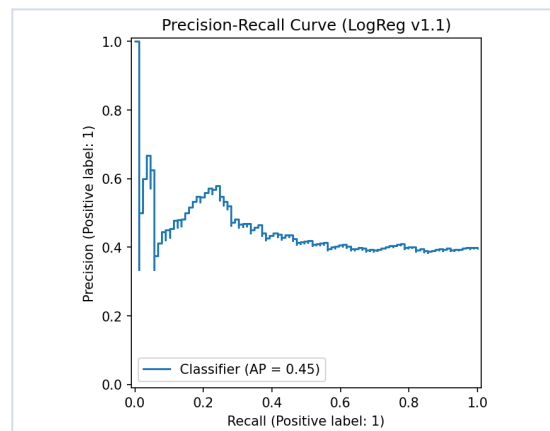
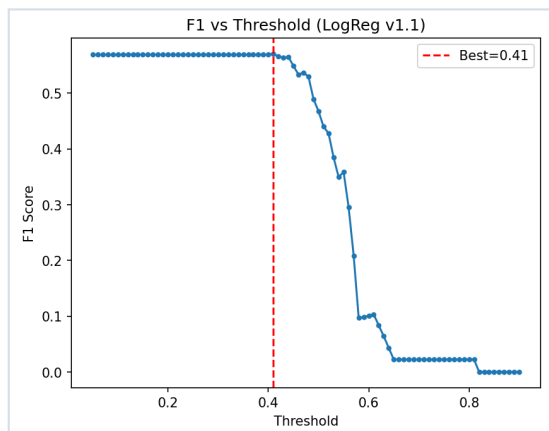
Final ROC AUC
Small but positive
ranking lift.

+0.03

AUC lift
Signal is weak, not
fake.

0.41

Decision threshold
Optimized for F1 / risk
alerting.



Why a weak model can be the correct model

In financial data, unusually strong performance is often a warning sign, not a success.

Domain reality

Markets are adaptive, noisy, and close to random over short horizons. A small edge is more credible than a dramatic score.

Modeling risk

High validation metrics may come from leakage, target proxies, time-order contamination, or overfitting.

v1.1 conclusion

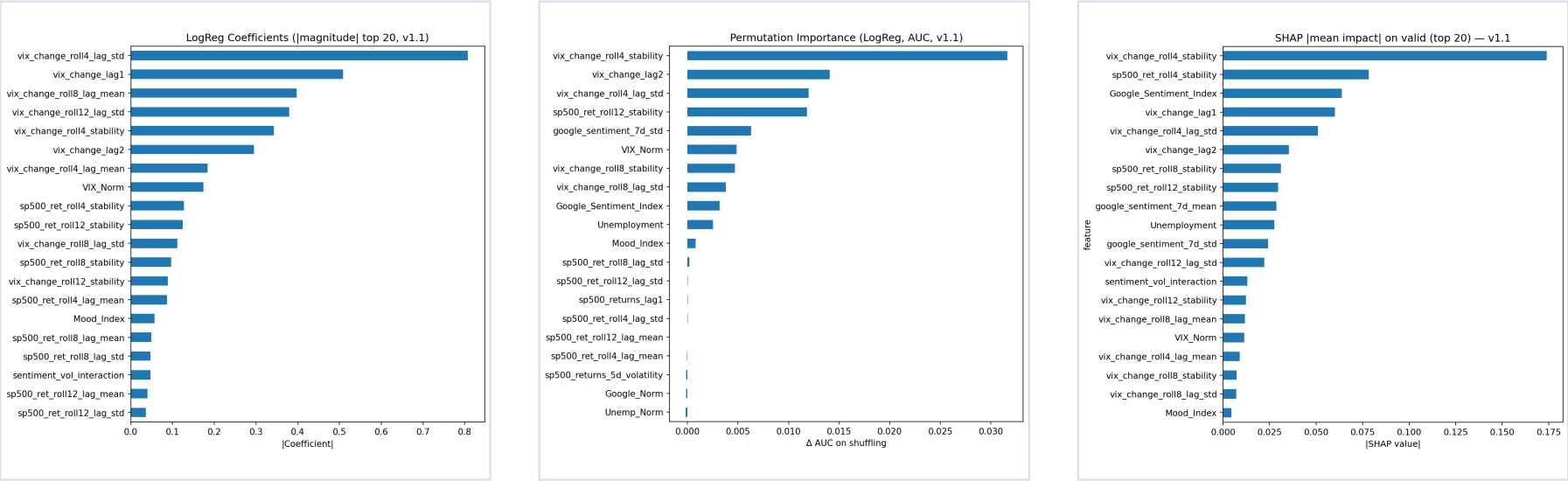
After leakage removal, lower performance is expected. The project is stronger because claims are conservative.

Interview message: The model became weaker after leakage removal — and that made it more trustworthy.

For this type of noisy financial problem, I would rather defend a modest, realistic model than present an inflated score that may be leaking future information.

Importance triangulation: volatility structure dominates

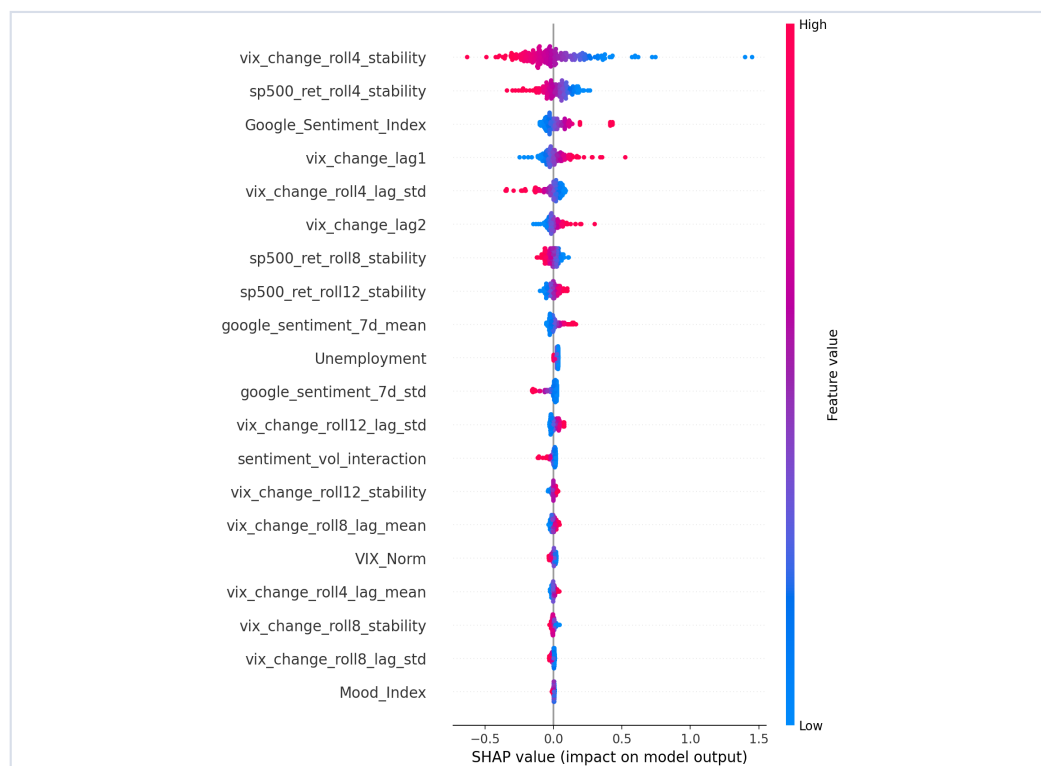
Coefficient magnitude, permutation importance, and SHAP converge on similar drivers.



Converging evidence: vix_change_rol4_stability and related VIX features dominate across multiple explanation methods. Sentiment contributes, but volatility structure is the main driver.

SHAP summary: predictions are interpretable

The model quantifies familiar market-stress mechanics rather than discovering a magic signal.



- VIX stability features have the widest SHAP spread and move predictions the most.
- Higher Google sentiment generally shifts risk downward, but the effect is smaller than volatility structure.
- Linear SHAP behavior is expected because the final estimator is Logistic Regression.
- Top drivers are explainable in market terms: volatility stress, market stability, and public search mood.

Gradio app wraps the artifact with guardrails

The app demonstrates product thinking, not just notebook modeling.

System design note: model artifact, metadata contract, and Gradio UI are decoupled (separation of concerns).

The screenshot shows a Gradio web interface for a model named 'Market Mood — Next-Week Outlook (Hotfix v1.1)'. The interface is dark-themed and includes a header with the title and a subtitle: 'Leakage-safe interface. Visible inputs are a small, interpretable subset; the rest are auto-filled from training medians. Explanations compare your inputs to a typical median baseline.' Below the header, there are tabs for 'Predict', 'Explain & Docs', and 'Diagnostics'. The 'Predict' tab is active. It contains a section for 'Inputs (visible features)' with a tip: 'Tip: baseline values are prefilled from training medians.' and a 'Best practice' section with three bullet points: 'all zeros = invalid / empty scenario', 'use real values for custom scenarios', and 'use Demo: nudge off baseline to prove the model moves'. There are three input fields for 'vix_change_rol4_stability', 'sp500_ret_rol4_stability', and 'Google_Sentiment_Index', each with a value of '0'. To the right of the input fields is a 'Prediction (score & prob)' section with a text area for the prediction and a 'Approx. 95% range (regression only)' section. Below the prediction section is an 'Explanations' section with a text area for the explanation. The interface also shows 'Model: v1.1-hotfix', 'Task: classification', 'Visible inputs: 3', and 'Invisible auto-filled features: 24'.

Artifact contract

.joblib stores the trained imputer, scaler, and model.

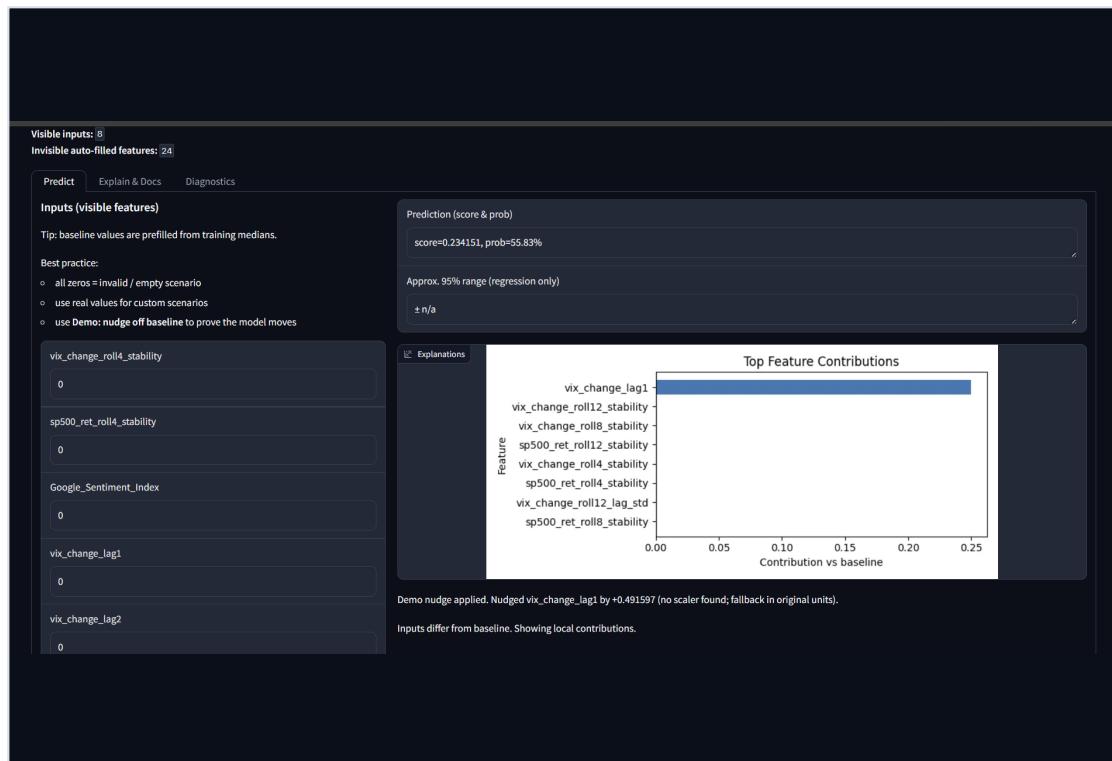
.json stores feature order, medians, visible inputs, blocked patterns, and version metadata.

App guardrails

Visible inputs are limited to 8 interpretable controls. The remaining 24 features are auto-filled from training medians. All-zero input is intentionally rejected.

Scenario changes produce explainable model movement

The interface proves that the saved model, metadata, and local explanation logic are aligned.



Scenario readout

Demo nudge changes the input vector and produces a new probability and feature-contribution chart.

Why this matters

The project is not only a notebook. It has a usable interface, local explanations, and basic deployment discipline.

Presentation proof

Screenshots show local execution, prediction, and explanation output. That is enough for portfolio proof of function.

Live demo: Hugging Face Spaces deployment available from the GitHub README.

Limitations make the project more credible

The model is for risk awareness and process demonstration, not financial advice.

What it can do

Support weekly monitoring, scenario review, and transparent discussion of market-stress signals.

What it cannot do

Guarantee market direction, replace human judgment, or support automated trading execution.

Next upgrades

Probability calibration
(better decision reliability)

Production-grade time-series validation pipeline

Multi-horizon prediction (1w, 2w, 4w)

Deployment monitoring and app usage testing

Credibility statement: The model is useful as a portfolio-grade demonstration of leakage-safe time-series ML, explainability, and deployment discipline.

Final assessment

This project reflects how I approach real-world ML problems where uncertainty is a part of the system.

What I focused on

- Leakage-safe, time-aware pipeline
- Realistic performance over inflated metrics
- Interpretable model, not a black box
- Working application not just a notebook

What this project proves

- I can debug and rebuild models when results are misleading
- I understand the difference between signal & leakage
- I can translate a model into a usable decision-support tool.
- I can structure ML work with clear constraints and guardrails.

How I think about ML systems

I treat modeling as a part of a system:

data → validation → model → explanation → usage

If one part is wrong (like leakage), the whole system becomes unreliable.

Positioning: I focus on building trustworthy ML systems under real-world constraints, not just optimizing metrics in isolation.